

```
# importing libraries
import numpy as np
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
plt.style.use('ggplot')

dataset = pd.read_csv('datasets/crop_yield.csv')
print(dataset.head())
```

	Crop	Crop_Year	Season	State	Area	Production \
0	Arecanut	1997	Whole Year	Assam	73814.0	56708
1	Arhar/Tur	1997	Kharif	Assam	6637.0	4685
2	Castor seed	1997	Kharif	Assam	796.0	22
3	Coconut	1997	Whole Year	Assam	19656.0	126905000
4	Cotton(lint)	1997	Kharif	Assam	1739.0	794

	Annual_Rainfall	Fertilizer	Pesticide	Yield
0	2051.4	7024878.38	22882.34	0.796087
1	2051.4	631643.29	2057.47	0.710435
2	2051.4	75755.32	246.76	0.238333
3	2051.4	1870661.52	6093.36	5238.051739
4	2051.4	165500.63	539.09	0.420909

Supervised and its a regression problem since continuous/non categorical values in output

```
dataset.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 19689 entries, 0 to 19688
Data columns (total 10 columns):
#   Column                Non-Null Count  Dtype
---  -
0   Crop                  19689 non-null  object
1   Crop_Year             19689 non-null  int64
2   Season                19689 non-null  object
3   State                 19689 non-null  object
4   Area                  19689 non-null  float64
5   Production            19689 non-null  int64
6   Annual_Rainfall       19689 non-null  float64
7   Fertilizer            19689 non-null  float64
8   Pesticide             19689 non-null  float64
9   Yield                 19689 non-null  float64
dtypes: float64(5), int64(2), object(3)
memory usage: 1.5+ MB

# check for null values
dataset.isnull().sum()
```

```
Crop          0
Crop_Year     0
Season        0
State         0
Area          0
Production    0
Annual_Rainfall 0
Fertilizer    0
Pesticide     0
Yield         0
dtype: int64
```

```
dataset.duplicated().sum()
# dataset.drop_duplicates(inplace=True)
# dataset.duplicated().sum()
```

```
np.int64(0)
```

```
dataset.shape
dataset.describe()
```

	Crop_Year	Area	Production	Annual_Rainfall \
count	19689.000000	1.968900e+04	1.968900e+04	19689.000000
mean	2009.127584	1.799266e+05	1.643594e+07	1437.755177
std	6.498099	7.328287e+05	2.630568e+08	816.909589
min	1997.000000	5.000000e-01	0.000000e+00	301.300000
25%	2004.000000	1.390000e+03	1.393000e+03	940.700000
50%	2010.000000	9.317000e+03	1.380400e+04	1247.600000
75%	2015.000000	7.511200e+04	1.227180e+05	1643.700000
max	2020.000000	5.080810e+07	6.326000e+09	6552.700000

	Fertilizer	Pesticide	Yield
count	1.968900e+04	1.968900e+04	19689.000000
mean	2.410331e+07	4.884835e+04	79.954009
std	9.494600e+07	2.132874e+05	878.306193
min	5.417000e+01	9.000000e-02	0.000000
25%	1.880146e+05	3.567000e+02	0.600000
50%	1.234957e+06	2.421900e+03	1.030000
75%	1.000385e+07	2.004170e+04	2.388889
max	4.835407e+09	1.575051e+07	21105.000000

data visualization

```
print(len(dataset['Crop'].unique())) #55
print(dataset['Crop'].unique())
```

```
55
```

```
['Arecanut' 'Arhar/Tur' 'Castor seed' 'Coconut ' 'Cotton(lint)'
'Dry chillies' 'Gram' 'Jute' 'Linseed' 'Maize' 'Mesta' 'Niger seed']
```

```
'Onion' 'Other Rabi pulses' 'Potato' 'Rapeseed &Mustard' 'Rice'
'Sesamum' 'Small millets' 'Sugarcane' 'Sweet potato' 'Tapioca'
'Tobacco'
'Turmeric' 'Wheat' 'Bajra' 'Black pepper' 'Cardamom' 'Coriander'
'Garlic'
'Ginger' 'Groundnut' 'Horse-gram' 'Jowar' 'Ragi' 'Cashewnut' 'Banana'
'Soyabean' 'Barley' 'Khesari' 'Masoor' 'Moong(Green Gram)'
'Other Kharif pulses' 'Safflower' 'Sannhamp' 'Sunflower' 'Urad'
'Peas & beans (Pulses)' 'other oilseeds' 'Other Cereals'
'Cowpea(Lobia)'
'Oilseeds total' 'Guar seed' 'Other Summer Pulses' 'Moth']
```

```
len(dataset['State'].unique())
```

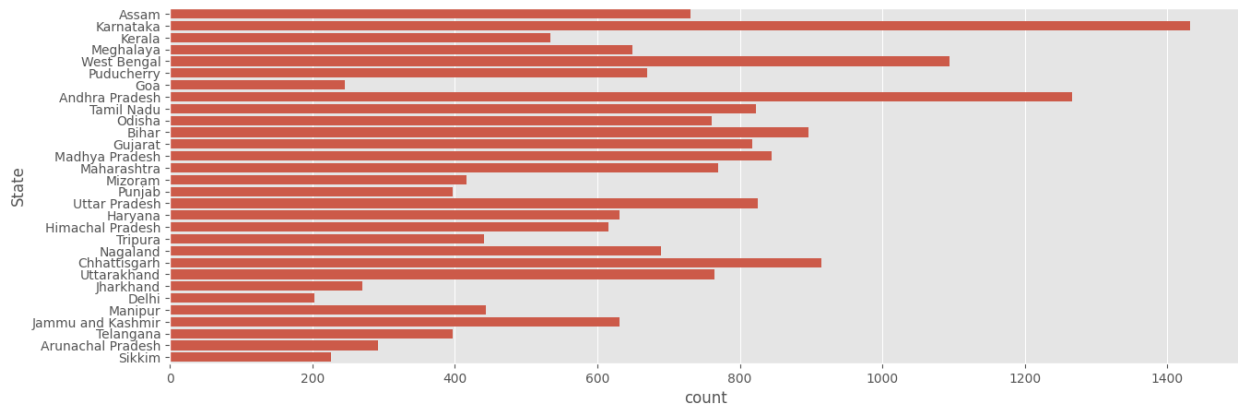
```
30
```

```
dataset['Season'] = dataset['Season'].str.strip().str.title()
```

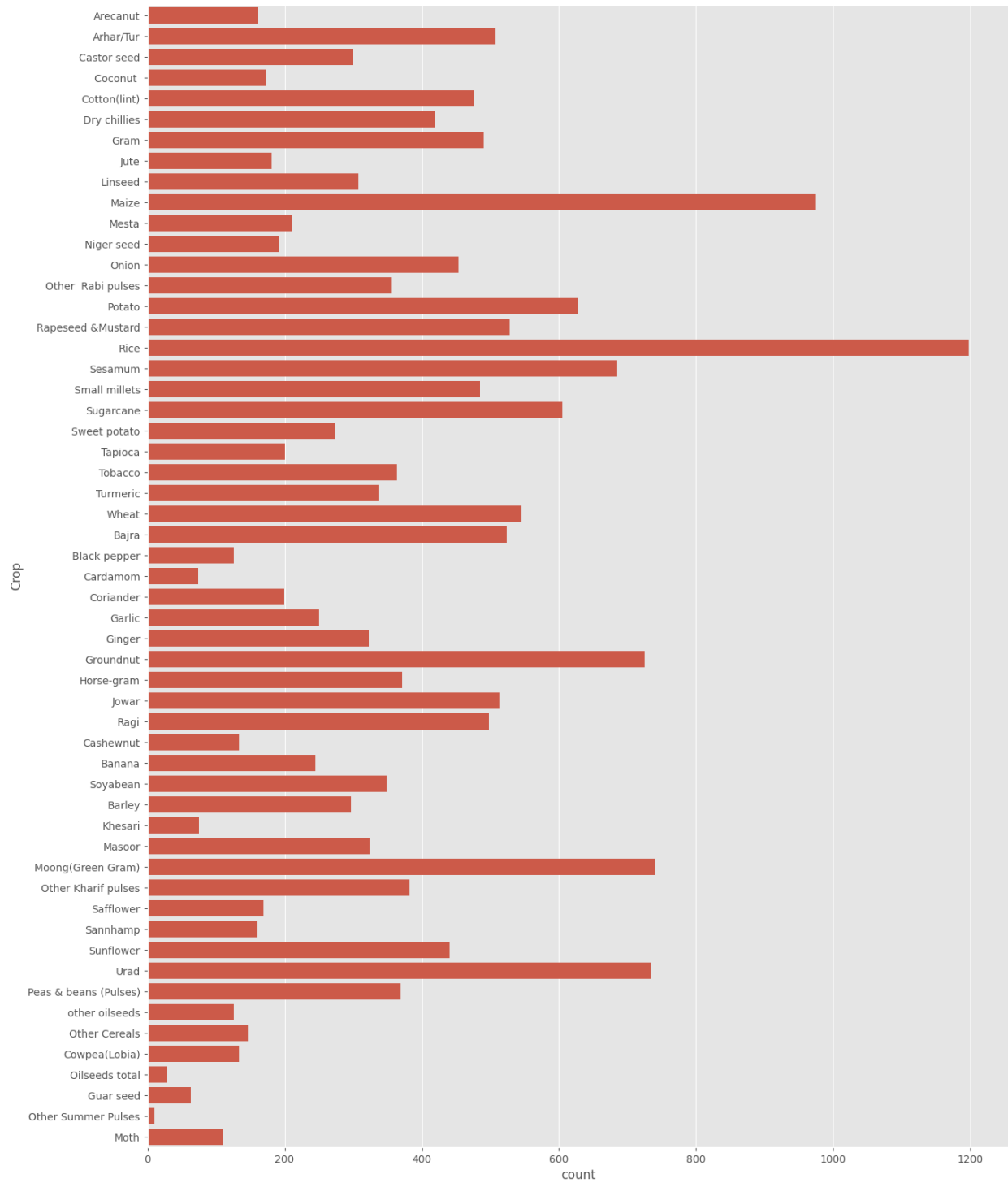
```
print(dataset['Season'].unique()) # 6 seasons
```

```
['Whole Year' 'Kharif' 'Rabi' 'Autumn' 'Summer' 'Winter']
```

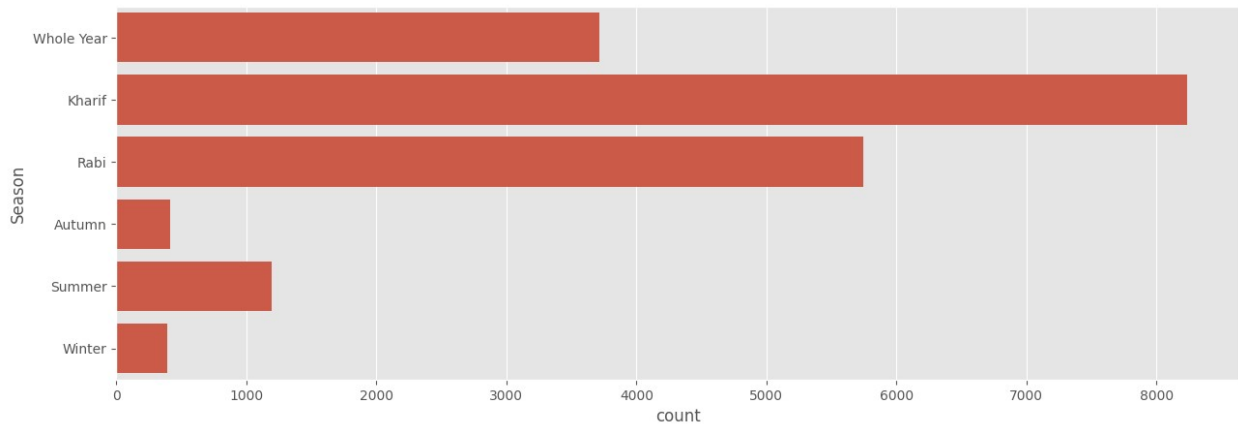
```
plt.figure(figsize=(15,5))
sns.countplot(y = dataset['State'])
plt.show()
```



```
plt.figure(figsize=(15,20))
sns.countplot(y = dataset['Crop'])
plt.show()
```



```
plt.figure(figsize=(15,5))
sns.countplot(y = dataset['Season'])
plt.show()
```



column transfers for less code so that we dont need to write code for every column s

```
dataset.columns
```

```
Index(['Crop', 'Crop_Year', 'Season', 'State', 'Area', 'Production',  
      'Annual_Rainfall', 'Fertilizer', 'Pesticide', 'Yield'],  
      dtype='object')
```

```
col = ['Crop_Year', 'Area', 'Production', 'Annual_Rainfall',  
      'Fertilizer', 'Pesticide', 'Crop', 'Season', 'State', 'Yield']
```

```
dataset = dataset[col]  
dataset.head()
```

	Crop_Year	Area	Production	Annual_Rainfall	Fertilizer
Pesticide \					
0	1997	73814.0	56708	2051.4	7024878.38
					22882.34
1	1997	6637.0	4685	2051.4	631643.29
					2057.47
2	1997	796.0	22	2051.4	75755.32
					246.76
3	1997	19656.0	126905000	2051.4	1870661.52
					6093.36
4	1997	1739.0	794	2051.4	165500.63
					539.09

	Crop	Season	State	Yield
0	Arecanut	Whole Year	Assam	0.796087
1	Arhar/Tur	Kharif	Assam	0.710435
2	Castor seed	Kharif	Assam	0.238333
3	Coconut	Whole Year	Assam	5238.051739
4	Cotton(lint)	Kharif	Assam	0.420909

```
X = dataset.drop('Yield', axis=1)  
y = dataset['Yield']
```

```

X.shape
(19689, 9)

y.shape
(19689,)

from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size =
0.25, random_state = 0 , shuffle=True)

from sklearn.preprocessing import OneHotEncoder , StandardScaler
from sklearn.compose import ColumnTransformer

ohe = OneHotEncoder(drop = 'first', handle_unknown='ignore')
scale = StandardScaler()

preprocessor = ColumnTransformer(
    transformers=[

        ('StandardScale', scale , [0,1,2,3,4,5]), #3 tuples
        ('OneHotEncoder', ohe, [6,7,8])
    ],
    remainder='passthrough'
)

print(dataset['Season'].unique())

['Whole Year' 'Kharif' 'Rabi' 'Autumn' 'Summer' 'Winter']

X_train = preprocessor.fit_transform(X_train)
X_test = preprocessor.transform(X_test)

preprocessor.get_feature_names_out(col[:-1])

array(['StandardScale__Crop_Year', 'StandardScale__Area',
      'StandardScale__Production', 'StandardScale__Annual_Rainfall',
      'StandardScale__Fertilizer', 'StandardScale__Pesticide',
      'OneHotEncoder__Crop_Arhar/Tur', 'OneHotEncoder__Crop_Bajra',
      'OneHotEncoder__Crop_Banana', 'OneHotEncoder__Crop_Barley',
      'OneHotEncoder__Crop_Black pepper',
      'OneHotEncoder__Crop_Cardamom',
      'OneHotEncoder__Crop_Cashewnut', 'OneHotEncoder__Crop_Castor
seed',
      'OneHotEncoder__Crop_Coconut ',
      'OneHotEncoder__Crop_Coriander',
      'OneHotEncoder__Crop_Cotton(lint)',
      'OneHotEncoder__Crop_Cowpea(Lobia)',
      'OneHotEncoder__Crop_Dry chillies',
      'OneHotEncoder__Crop_Garlic',
      'OneHotEncoder__Crop_Ginger', 'OneHotEncoder__Crop_Gram',

```

```

seed', 'OneHotEncoder__Crop_Groundnut', 'OneHotEncoder__Crop_Guar
seed', 'OneHotEncoder__Crop_Horse-gram', 'OneHotEncoder__Crop_Jowar',
'OneHotEncoder__Crop_Jute', 'OneHotEncoder__Crop_Khesari',
'OneHotEncoder__Crop_Linseed', 'OneHotEncoder__Crop_Maize',
'OneHotEncoder__Crop_Masoor', 'OneHotEncoder__Crop_Mesta',
'OneHotEncoder__Crop_Moong(Green Gram)',
'OneHotEncoder__Crop_Moth', 'OneHotEncoder__Crop_Niger seed',
'OneHotEncoder__Crop_Oilseeds total',
'OneHotEncoder__Crop_Onion',
'OneHotEncoder__Crop_Other Rabi pulses',
'OneHotEncoder__Crop_Other Cereals',
'OneHotEncoder__Crop_Other Kharif pulses',
'OneHotEncoder__Crop_Other Summer Pulses',
'OneHotEncoder__Crop_Peas & beans (Pulses)',
'OneHotEncoder__Crop_Potato', 'OneHotEncoder__Crop_Ragi',
'OneHotEncoder__Crop_Rapeseed & Mustard',
'OneHotEncoder__Crop_Rice', 'OneHotEncoder__Crop_Safflower',
'OneHotEncoder__Crop_Sannhamp', 'OneHotEncoder__Crop_Sesamum',
'OneHotEncoder__Crop_Small millets',
'OneHotEncoder__Crop_Soyabean',
'OneHotEncoder__Crop_Sugarcane',
'OneHotEncoder__Crop_Sunflower',
'OneHotEncoder__Crop_Sweet potato',
'OneHotEncoder__Crop_Tapioca',
'OneHotEncoder__Crop_Tobacco', 'OneHotEncoder__Crop_Turmeric',
'OneHotEncoder__Crop_Urad', 'OneHotEncoder__Crop_Wheat',
'OneHotEncoder__Crop_other oilseeds',
'OneHotEncoder__Season_Kharif', 'OneHotEncoder__Season_Rabi',
'OneHotEncoder__Season_Summer', 'OneHotEncoder__Season_Whole
Year',
'OneHotEncoder__Season_Winter',
'OneHotEncoder__State_Arunachal Pradesh',
'OneHotEncoder__State_Assam', 'OneHotEncoder__State_Bihar',
'OneHotEncoder__State_Chhattisgarh',
'OneHotEncoder__State_Delhi',
'OneHotEncoder__State_Goa', 'OneHotEncoder__State_Gujarat',
'OneHotEncoder__State_Haryana',
'OneHotEncoder__State_Himachal Pradesh',
'OneHotEncoder__State_Jammu and Kashmir',
'OneHotEncoder__State_Jharkhand',
'OneHotEncoder__State_Karnataka',
'OneHotEncoder__State_Kerala',
'OneHotEncoder__State_Madhya Pradesh',
'OneHotEncoder__State_Maharashtra',
'OneHotEncoder__State_Manipur',
'OneHotEncoder__State_Meghalaya',
'OneHotEncoder__State_Mizoram',
'OneHotEncoder__State_Nagaland', 'OneHotEncoder__State_Odisha',

```

```

        'OneHotEncoder__State_Puducherry',
'OneHotEncoder__State_Punjab',
        'OneHotEncoder__State_Sikkim', 'OneHotEncoder__State_Tamil
Nadu',
        'OneHotEncoder__State_Telangana',
'OneHotEncoder__State_Tripura',
        'OneHotEncoder__State_Uttar Pradesh',
        'OneHotEncoder__State_Uttarakhand',
        'OneHotEncoder__State_West Bengal'], dtype=object)

```

```

from sklearn.linear_model import LinearRegression , Lasso , Ridge
from sklearn.neighbors import KNeighborsRegressor
from sklearn.tree import DecisionTreeRegressor
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import median_absolute_error , r2_score ,
accuracy_score

```

```

models = {
    'Linear Regression': LinearRegression(),
    'Lasso' : Lasso(),
    'Ridge' : Ridge(),
    'Decision Tree' : DecisionTreeRegressor(),
    # 'Random Forest' : RandomForestRegressor(),
    'KNN' : KNeighborsRegressor()
}

```

```

}

```

```

for name , model_names in models.items():
    model_names.fit(X_train , y_train)
    y_pred = model_names.predict(X_test)
    print(f'{name} The mean absolute error :
{median_absolute_error(y_test, y_pred)} , The r2
score{r2_score(y_test, y_pred)}')

```

```

Linear Regression The mean absolute error : 19.85017569792159 , The r2
score0.854350554771411

```

```

Lasso The mean absolute error : 11.41151683730862 , The r2
score0.8506379457943178

```

```

Ridge The mean absolute error : 20.314373672231245 , The r2
score0.8526515274374293

```

```

Decision Tree The mean absolute error : 0.10892857099999997 , The r2
score0.9491492670015277

```

```

KNN The mean absolute error : 0.1452139864 , The r2
score0.9435937523627809

```

```

knn = KNeighborsRegressor()
knn.fit(X_train, y_train)
knn.predict(X_test)

```



```
array([42.69199261,  5.16725926,  0.7023141 , ...,  5.97873333,
        0.33357143,  1.75602015])
```

predict system

```
dataset.columns
```

```
Index(['Crop_Year', 'Area', 'Production', 'Annual_Rainfall',
       'Fertilizer',
       'Pesticide', 'Crop', 'Season', 'State', 'Yield'],
      dtype='object')
```

```
def prediction(Crop_Year, Area, Production, Annual_Rainfall,
               Fertilizer, Pesticide, Crop, Season, State):
    features = np.array([Crop_Year, Area, Production, Annual_Rainfall,
                        Fertilizer, Pesticide, Crop, Season, State])
    transform_features = preprocessor.transform(features)
    predicted_yield = knn.predict(transform_features).reshape(-1,1)
    return predicted_yield[0][0]
```

```
print(prediction(1997,73814,56708,2051,7024878,22882,"Arecanut","Whole
Year","Assam"))
```

```
-----
-----
```

```
TypeError                                Traceback (most recent call
last)
```

```
File c:\Python312\Lib\site-packages\sklearn\base.py:420, in
BaseEstimator._check_n_features(self, X, reset)
```

```
    419 try:
--> 420     n_features = _num_features(X)
    421 except TypeError as e:
```

```
File c:\Python312\Lib\site-packages\sklearn\utils\validation.py:345,
in _num_features(X)
```

```
    344     message += f" with shape {X.shape}"
--> 345     raise TypeError(message)
    346 return X.shape[1]
```

```
TypeError: Unable to find the number of features from X of type
numpy.ndarray with shape (9,)
```

The above exception was the direct cause of the following exception:

```
ValueError                                Traceback (most recent call
last)
```

```
Cell In[250], line 7
```

```
    4     predicted_yield =
knn.predict(transform_features).reshape(-1,1)
    5     return predicted_yield[0][0]
```

```

----> 7
print(prediction(1997,73814,56708,2051,7024878,22882,"Arecanut","Whole
Year","Assam"))

```

```

Cell In[250], line 3, in prediction(Crop_Year, Area, Production,
Annual_Rainfall, Fertilizer, Pesticide, Crop, Season, State)
      1 def prediction(Crop_Year, Area,Production, Annual_Rainfall,
Fertilizer, Pesticide, Crop, Season, State):
      2     features = np.array([Crop_Year, Area,Production,
Annual_Rainfall, Fertilizer, Pesticide, Crop, Season, State])
----> 3     transform_features = preprocessor.transform(features)
      4     predicted_yield =
knn.predict(transform_features).reshape(-1,1)
      5     return predicted_yield[0][0]

```

```

File c:\Python312\Lib\site-packages\sklearn\utils\_set_output.py:313,
in _wrap_method_output.<locals>.wrapped(self, X, *args, **kwargs)
    311 @wraps(f)
    312 def wrapped(self, X, *args, **kwargs):
--> 313     data_to_wrap = f(self, X, *args, **kwargs)
    314     if isinstance(data_to_wrap, tuple):
    315         # only wrap the first output for cross decomposition
    316         return_tuple = (
    317             _wrap_data_with_container(method, data_to_wrap[0],
X, self),
    318             *data_to_wrap[1:],
    319         )

```

```

File c:\Python312\Lib\site-packages\sklearn\compose\
_column_transformer.py:1069, in ColumnTransformer.transform(self, X,
**params)
    1065         raise ValueError(f"columns are missing: {diff}")
    1066     else:
    1067         # ndarray was used for fitting or transforming, thus we
only
    1068         # check that n_features_in_ is consistent
-> 1069     self._check_n_features(X, reset=False)
    1071 if _routing_enabled():
    1072     routed_params = process_routing(self, "transform",
**params)

```

```

File c:\Python312\Lib\site-packages\sklearn\base.py:423, in
BaseEstimator._check_n_features(self, X, reset)
    421 except TypeError as e:
    422     if not reset and hasattr(self, "n_features_in_"):
--> 423         raise ValueError(
    424             "X does not contain any features, but "
    425             f"{self.__class__.__name__} is expecting "
    426             f"{self.n_features_in_} features"
    427         ) from e

```

```
428     # If the number of features is not defined and reset=True,  
429     # then we skip this check  
430     return
```

ValueError: X does not contain any features, but ColumnTransformer is expecting 9 features